

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 1: Constraint-Based Problem Solving**

**COURSE NAME: COMPUTER VISION**

**COURSE CODE: ITA0515**

**Slot B**

---

**COURSE OUTCOME COVERED**

**CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)**

**Assessment Tool Weightage: 40%**

---

**QUESTIONS:5**

**Total Marks 50**

**Duration :60 Minutes**

**Annexure A**

**Constraint-Based Problem Solving**

---

**Code of Conduct:**

***I S.Jeswanth Kumar (192321056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.***

**Signature of the student: S.Jeswanth Kumar**

**Rubrics for Evaluation**

| Criteria                | Weightage (%) | Level 4 (Exemplary)                                                            | Level 3 (Proficient)                                  | Level 2 (Developing)                          | Level 1 (Beginning)                      | Marks(100) |
|-------------------------|---------------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|------------|
| Problem Understanding   | 25%           | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |            |
| Constraint Analysis     | 25%           | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |            |
| Solution Development    | 25%           | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |            |
| Application of Concepts | 15%           | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |            |
| Justification           | 10%           | Provides strong justification for decisions with logical reasoning             | Provides justification with some clarity              | Weak or unclear justification                 | No justification provided                |            |

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

#### 1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

#### 2. Automated Video Surveillance with Alert System

A security firm needs a video surveillance system that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

#### 3. OCR System for Low-Quality Documents

A digitization company needs an OCR system that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

#### 4. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

#### 5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

## ANSWERS

1. A low-power face recognition system can be designed using an efficient OpenCV pipeline:

**Live Camera → Preprocessing → Face Detection → Face Alignment/Feature Extraction → Face Recognition → Output**

### 1. Image Acquisition:

Capture live video using `cv2.VideoCapture()`. The camera resolution can be reduced to 640×480 to decrease CPU and memory usage while maintaining sufficient face details.

### 2. Preprocessing:

Convert frames from BGR to grayscale using `cv2.cvtColor()`. Apply histogram equalization or CLAHE to improve visibility under varying lighting conditions. Resize the frame before detection to reduce computational cost.

### 3. Face Detection:

Use an OpenCV Haar Cascade classifier or a lightweight DNN face detector to locate faces. Haar Cascades are suitable for low-power devices because they require relatively low computational resources.

### 4. Feature Extraction and Recognition:

Detect the facial region and extract discriminative facial features. For a predefined dataset, an OpenCV face recognizer such as LBPH can be used. LBPH is suitable for edge devices because it is lightweight and performs reasonably well under moderate illumination changes.

### 5. Output:

Display the detected face using `cv2.rectangle()` and show the recognized person's name and confidence using `cv2.putText()`.

```
face = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
recognizer = cv2.face.LBPHFaceRecognizer_create()
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face.detectMultiScale(gray, 1.1, 5)
    for (x, y, w, h) in faces:
        label, confidence = recognizer.predict(gray[y:y+h, x:x+w])
        cv2.rectangle(frame, (x,y), (x+w,y+h), (255,0,0), 2)
        cv2.putText(frame, str(label), (x,y-10),
```

```

        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,0), 2)
cv2.imshow("Face Recognition", frame)
if cv2.waitKey(1) & 0xFF == 27:
    break
cap.release()
cv2.destroyAllWindows()

```

Constraint justification: Frame resizing and lightweight LBPH reduce CPU and memory consumption. Grayscale processing reduces computation, while histogram enhancement improves performance under different lighting conditions. Processing only detected face regions further improves real-time performance.

**2.** The surveillance system can be implemented using the following pipeline:

**CCTV Feed → Frame Acquisition → Noise Reduction → Background/Motion Detection → Feature Analysis → Activity Verification → Alert → Display/Storage**

### **1. Video Acquisition:**

Use `cv2.VideoCapture()` to read standard CCTV streams or video files. Frames can be resized to reduce processing time.

### **2. Preprocessing:**

Convert frames to grayscale using `cv2.cvtColor()` and apply Gaussian blur using `cv2.GaussianBlur()` to reduce camera noise.

### **3. Motion Detection:**

Use background subtraction methods such as `cv2.createBackgroundSubtractorMOG2()` or frame differencing to identify moving objects.

### **4. Feature and Activity Analysis:**

Extract contours using `cv2.findContours()`. Small contours caused by noise are removed using an area threshold. Bounding boxes can be generated using `cv2.boundingRect()`. Motion can then be analyzed across consecutive frames to identify unusual activities.

### **5. Alert Generation:**

If significant motion remains inside a predefined restricted region for a specified number of consecutive frames, the system generates an alert. The event can be displayed using `cv2.putText()` and recorded for later analysis.

Constraint justification: Gaussian filtering and contour-area filtering reduce false positives caused by noise. MOG2 adapts to changing backgrounds, making it more suitable for dynamic environments. Region-of-interest processing reduces unnecessary computation. Frame resizing and processing only significant motion regions help maintain real-time performance.

**3.** An efficient OCR system for low-quality documents can be designed as:

**Scanned Image → Preprocessing → Noise Removal → Contrast Enhancement → Binarization → Segmentation → Morphological Processing → OCR → Text Output**

**1. Image Acquisition:**

Read scanned documents using `cv2.imread()` and process multiple images sequentially for batch processing.

**2. Preprocessing:**

Convert the image to grayscale using `cv2.cvtColor()`. Resize the image using `cv2.resize()` when the original resolution is too low.

**3. Noise Removal:**

Apply `cv2.GaussianBlur()` or median filtering to remove scanning noise and small unwanted variations.

**4. Contrast Enhancement:**

Apply histogram equalization or CLAHE to improve text visibility under uneven illumination.

**5. Segmentation:**

Convert the enhanced image into a binary image using adaptive thresholding such as `cv2.adaptiveThreshold()`. Adaptive thresholding is useful when illumination varies across the document.

**6. Morphological Processing:**

Use `cv2.morphologyEx()` with operations such as opening and closing to remove small noise and connect broken characters.

**7. Text Extraction:**

Pass the processed image to an OCR engine. The extracted text can then be stored in a text file or database.

Grayscale conversion and resizing reduce computational requirements. Adaptive thresholding handles varying illumination, while denoising and morphological operations improve character separation. Processing documents sequentially or in batches allows efficient large-scale digitization.

**4.** The facial expression recognition system can use the following pipeline:

**Mobile Camera → Frame Preprocessing → Face Detection → Face Region Extraction → Expression Analysis → Classification → Display**

**1. Image Acquisition:**

Capture frames from the mobile camera. Reduce the resolution or process every alternate frame to reduce CPU usage while maintaining real-time performance.

**2. Preprocessing:**

Convert frames to grayscale using `cv2.cvtColor()`. Apply histogram equalization or CLAHE to reduce the effect of lighting variations.

**3. Face Detection:**

Use an OpenCV Haar Cascade or lightweight DNN model to detect the face. The detected region is extracted using its bounding coordinates.

**4. Expression Analysis:**

Normalize the face region by resizing it to a fixed size. Facial features such as the mouth, eyes, and eyebrows can be analyzed, or a lightweight trained classifier can classify the expressions as happy, sad, or neutral.

**5. Output:**

Display the detected expression using `cv2.putText()` and draw the face boundary using `cv2.rectangle()`.

Constraint justification: Lightweight face detection, grayscale processing, reduced frame resolution, and frame skipping decrease CPU usage. Face normalization improves consistency when the head moves slightly. CLAHE improves robustness under lighting changes. A lightweight classifier provides a suitable balance between recognition accuracy and mobile performance.

**5.** A smart traffic monitoring system can be designed using the following pipeline:

**Live CCTV → Frame Acquisition → Preprocessing → Vehicle Detection → Tracking → Counting → Statistics → Visualization**

**1. Video Handling:**

Capture the live CCTV stream using `cv2.VideoCapture()`. Frames can be resized to an appropriate resolution to maintain real-time performance.

**2. Preprocessing:**

Convert frames to grayscale and apply Gaussian blur to reduce noise. A region of interest can be selected so that only the road area is processed.

### **3. Vehicle Detection:**

Detect moving vehicles using background subtraction such as `cv2.createBackgroundSubtractorMOG2()`. Contours can be obtained using `cv2.findContours()`, and bounding boxes can be generated using `cv2.boundingRect()`.

### **4. Vehicle Tracking:**

Assign an ID to each detected vehicle and track its position across consecutive frames using centroid-based tracking. The movement of each vehicle is determined from its changing centroid coordinates.

### **5. Vehicle Counting:**

A virtual counting line is placed across the road. When a tracked vehicle crosses the line in the required direction, the vehicle counter is incremented.

### **6. Visualization:**

Use `cv2.rectangle()` to draw bounding boxes, `cv2.putText()` to display vehicle IDs and counts, and `cv2.line()` to display the counting line.

```
cv2.rectangle(frame, (x, y), (x+w, y+h), (255,0,0), 2)
```

```
cv2.putText(frame, "Vehicle", (x, y-10),
```

```
    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
```

```
cv2.line(frame, (0, 300), (640, 300), (0,0,255), 2)
```

Constraint justification: Region-of-interest processing and frame resizing reduce computational load. Tracking prevents repeatedly counting the same vehicle. A virtual line provides reliable counting, while contour filtering removes small objects and noise. Continuous frame processing and lightweight detection allow the system to operate with minimal delay.